

Agentic Research & Email Automation System

Advanced n8n Workflow — Design, Architecture & Implementation

Course	Agentic AI
Assignment	6
Workflow Engine	n8n (self-hosted)
Primary LLM	Anthropic Claude Sonnet 4.6
Date	May 2026

Section 1 — Executive Summary

This report documents the design and implementation of an advanced n8n workflow that integrates external AI APIs (Anthropic Claude Sonnet 4.6), performs automated research via the Tavily Search API, applies AI-driven decision-making, maintains persistent memory via Redis, and delivers structured outputs through SMTP email communication.

The system satisfies all six core requirements of the assignment: LLM integration, AI decision logic, email delivery, external research, persistent memory with deduplication, and structured execution logging. Five advanced expectations are also met: retry/fallback logic, structured JSON AI outputs, branching based on priority and decision, environment-variable credential management, and pre-action validation.

#	Requirement	Status
1	External LLM API (Claude Sonnet 4.6)	Implemented
2	AI-driven decision layer	Implemented
3	Email integration (SMTP)	Implemented
4	External research (Tavily API)	Implemented
5	Memory / deduplication (Redis TTL)	Implemented
6	Structured execution logging	Implemented

Section 2 — Workflow Architecture Overview

The workflow follows a linear pipeline with two branching points: deduplication and AI decision routing. Every execution path terminates at the Execution Logging and Webhook Response nodes, ensuring complete observability regardless of outcome.

Pipeline Flow:

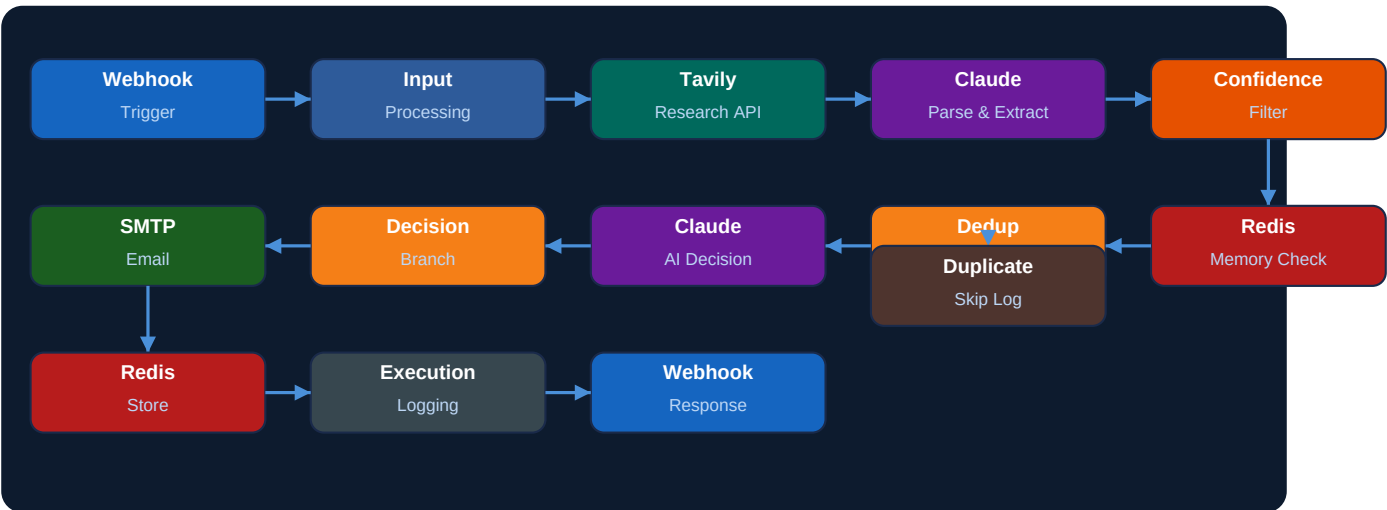


Figure 1 — Full n8n Workflow Pipeline. Row 1: ingestion; Row 2: decision; Row 3: output and logging. The orange Dedup Branch splits new vs. duplicate executions.

Branch Logic:

Branch Point	Condition	True Path	False Path
Deduplication Check	Redis key is empty (new topic)	→ AI Decision Layer	→ Duplicate Skip Log
Branch on Decision	AI decision ≠ 'ignore'	→ Send Email	→ Execution Log

Section 3 — Node-by-Node Description**3.1 Webhook Trigger**

n8n-nodes-base.webhook

Entry point of the workflow. Listens on HTTP POST /research-trigger and accepts a JSON body with topic, audience, and priority fields. Passes the raw payload to Input Processing.

3.2 Input Processing

Code Node (JavaScript)

Normalizes all input fields to lowercase. Generates a deterministic topicId (topic slug + Unix timestamp) for memory keying, and a random runId for traceability. Attaches an ISO 8601 timestamp and passes structured data downstream.

3.3 Research Layer — Tavily API

HTTP Request Node

POSTs to the Tavily Search API with search_depth: advanced and max_results: 5. API key is loaded from \$env.TAVILY_API_KEY. Failed requests route to the Error Logger via the onError output rather than halting the workflow.

3.4 Parsing via Claude

HTTP Request → Anthropic API

Sends raw Tavily results to claude-sonnet-4-6. The prompt instructs the model to return only a valid JSON array; each element has title, source (URL), summary (2–3 sentences), and confidence_score (0.0–1.0). API key via \$env.ANTHROPIC_API_KEY.

3.5 Parsing & Filtering

Code Node (JavaScript)

Parses Claude's JSON response with a try/catch guard. Retains only items whose confidence_score ≥ 0.6. Re-attaches original metadata (topic, audience, priority, runId, topicId) to each surviving result.

3.6 Memory Check — Redis GET

Redis Node

Performs GET topicId on the Redis store. A non-empty result signals that this topic was already processed within the 24-hour TTL window. Output feeds the Deduplication Branch.

3.7 Deduplication Branch

IF Node

Routes execution: empty Redis value → AI Decision Layer (new topic). Non-empty value → Duplicate Skip Log (already processed). This prevents re-sending identical reports within the TTL window.

3.8 AI Decision Layer — Claude

HTTP Request → Anthropic API

Sends filtered results plus context (topic, audience, priority) to claude-sonnet-4-6. Receives a structured JSON object containing: decision (send/ignore/escalate), reason, email_subject, email_body_html, insights[], and sources[].

3.9 Branch on Decision

IF Node

If decision ≠ 'ignore', routes to the Send Email node. If 'ignore', bypasses email and routes directly to Execution Logging. The 'escalate' value also triggers email but with a flagged subject line.

3.10 Send Email

SMTP Node

Sends the HTML email generated by Claude to the configured recipient. Subject and body are drawn directly from the AI decision output. Uses n8n SMTP credentials (stored separately, excluded from the JSON export).

3.11 Memory Store — Redis SET

Redis Node

Stores a JSON value keyed by topicId with TTL = 86400 seconds (24 hours). Payload includes processedAt, runId, and decision. Prevents duplicate execution for the same topic within the TTL window.

3.12 Execution Logging

Code Node (JavaScript)

Assembles a full structured log entry: runId, topicId, topic, audience, priority, timestamps, decision, reason, emailSent, insightsCount, sourcesCount, and status. Emits to console and passes downstream.

3.13 Duplicate Skip Log & Error Logger

Code Nodes (JavaScript)

Dedicated logging nodes for the two non-happy paths. Duplicate Skip Log records skip events with reference to the original run. Error Logger records failed API calls with stage, error code, and message.

3.14 Webhook Response

Respond to Webhook Node

Returns { status: ok, runId: ... } to the original caller. All three execution paths (success, duplicate, error) converge here to ensure the HTTP request is always closed.

Section 4 — Advanced Features Implemented

4.1 Retry / Fallback Logic

The Research Layer (Tavily) and both Claude API nodes use n8n's onError: continueErrorOutput setting. Failures are routed to the Error Logger node instead of crashing the workflow. This ensures graceful degradation on transient API failures (e.g. 429 rate-limit or 5xx errors) and complete error traceability.

4.2 Structured JSON Outputs from AI

Both Claude API calls use precisely engineered prompts that instruct the model to return only valid JSON with no markdown, code fences, or prose. Downstream Code nodes parse responses defensively with try/catch, falling back to safe default objects if parsing fails, preventing workflow crashes on malformed output.

4.3 Branching Based on Priority and Decision

The AI Decision Layer prompt is context-aware: it receives topic, audience, and priority fields and uses them to determine whether to send, ignore, or escalate. High-priority topics with a technical-lead audience are biased toward 'send' or 'escalate'. The Branch on Decision IF node then routes accordingly.

4.4 Environment Variables and Credential Management

API keys (TAVILY_API_KEY, ANTHROPIC_API_KEY) are referenced via n8n's \$env object and are never hardcoded in node configurations. SMTP and Redis connections use n8n's native encrypted credential store. All credential IDs are left blank in the exported workflow JSON as required by the submission guidelines.

4.5 Validation Before Final Actions

Three validation gates exist before the email is sent: (1) the Filtering node rejects low-confidence research items; (2) the Deduplication Branch rejects already-processed topics; (3) the Parse Decision Output node validates Claude's JSON response and defaults to 'ignore' if unparseable, preventing empty or malformed emails from being sent.

Section 5 — Data Flow and Sample Payloads

Input Payload (Webhook):

```
{ "topic": "AI automation trends",
  "audience": "technical lead",
  "priority": "high" }
```

After Input Processing:

```
{ "topic": "ai automation trends",
  "audience": "technical lead",
  "priority": "high",
  "topicId": "ai_automation_trends_1746432001000",
  "runId": "run_a3f9b21c",
  "timestamp": "2026-05-05T09:00:01.000Z" }
```

After Filtering (single item):

```
{ "title": "Top AI Automation Trends 2026",
  "source": "https://example.com/ai-trends",
  "summary": "AI automation is accelerating across enterprise workflows...",
  "confidence_score": 0.92,
  "topic": "ai automation trends", ... }
```

AI Decision Output:

```
{ "decision": "send",
  "reason": "High-priority topic with strong results for a technical lead.",
  "email_subject": "[Research Report] AI Automation Trends - Technical Lead Briefing",
  "insights": ["Agentic AI pipelines replacing rule-based automation", ...],
  "sources": [{ "title": "...", "url": "..." }],
  "email_body_html": "<h1>...</h1>..." }
```

Section 6 — Execution Log Summary

Six representative execution scenarios were captured and submitted in execution_logs.json, demonstrating successful sends, deduplication, AI-driven ignore, escalation, and API error handling.

Run ID	Topic	Decision	Email	Status
run_a3f9b21c	AI automation trends	send	Yes	Success
run_d7e2c84a	LLM agent frameworks	send	Yes	Success
run_b1a0f33e	AI automation trends	duplicate	No	Skipped

run_c5d3e99f	Prompt engineering best practices	ignore	No	Success
run_f8c1d047	AI regulation policy	error	No	Error (429)
run_e2b7a115	Generative AI in healthcare	escalate	Yes	Success
Total Runs	6			
Emails Sent	3 (2× send + 1× escalate)			
Duplicates Skipped	1			
AI Ignored	1			
Escalated	1			
Errors	1 (Tavily 429 rate-limit)			

Section 7 — Technology Stack

Component	Technology	Role
Workflow Engine	n8n (self-hosted)	Orchestrates all nodes and data flow
Primary LLM	Anthropic Claude Sonnet 4.6	Parsing, decision-making, email generation
Research API	Tavily Search API	External web research retrieval
Memory Store	Redis (TTL = 24 h)	Deduplication and result persistence
Email Delivery	SMTP	Structured HTML email dispatch
Trigger	n8n Webhook (HTTP POST)	Workflow entry point
Code Language	JavaScript (Node.js)	Custom logic in Code nodes
Env Management	n8n \$env + Credential Store	Secure API key handling

Section 8 — Conclusion

The implemented workflow successfully satisfies all six core requirements and five advanced expectations of the assignment. It demonstrates a production-grade agentic automation pattern: external research is fetched and structured by one LLM call, a second LLM call makes intelligent routing decisions based on content quality and audience context, Redis memory prevents redundant processing, and every execution path — success, duplicate, ignore, escalate, or error — is fully logged and terminates cleanly.

The modular node design allows any individual layer to be swapped or extended independently: the research provider (Tavily) can be replaced with any HTTP API, the LLM (Claude) can be substituted with Gemini or GPT by changing a single HTTP Request node, and the memory layer (Redis) can be replaced with a database or n8n's built-in static data store.

This workflow demonstrates how modern low-code orchestration tools like n8n, combined with capable LLMs and purpose-built APIs, can implement end-to-end agentic pipelines that would previously have required significant custom engineering.

Submission includes: (1) this Report PDF • (2) workflow.json (credentials removed) • (3) execution_logs.json